

# HE-SecureNet: An Efficient and Usable Framework for Model Training via Homomorphic Encryption\*

Thomas Schneider<sup>✉</sup>

TU Darmstadt  
Darmstadt, Germany  
schneider@crypto.cs.tu-darmstadt.de

Huan-Chih Wang<sup>✉\*</sup>

National Taiwan University  
Taipei, Taiwan  
whcjimmy@cmlab.csie.ntu.edu.tw

Hossein Yalame<sup>✉</sup>

TU Darmstadt  
Darmstadt, Germany  
yalame@crypto.cs.tu-darmstadt.de

## Abstract

Energy-efficient edge devices are essential for the widespread deployment of machine learning (ML) services. However, their limited computational capabilities make local model training infeasible. While cloud-based training offers a scalable alternative, it raises serious privacy concerns when sensitive data is outsourced. Homomorphic Encryption (HE) enables computation directly on encrypted data and has emerged as a promising solution to this privacy challenge. Yet, current HE-based training frameworks face several shortcomings: they often lack support for complex models and non-linear functions, struggle to train over multiple epochs, and require cryptographic expertise from end users.

We present *HE-SecureNet*, a novel framework for privacy-preserving model training on encrypted data in a single-client-server setting, using hybrid HE cryptosystems. Unlike prior HE-based solutions, *HE-SecureNet* supports advanced models such as Convolutional Neural Networks and handles non-linear operations including ReLU, Softmax, and MaxPooling. It introduces a level-aware training strategy that eliminates costly ciphertext level alignment across epochs. Furthermore, *HE-SecureNet* automatically converts ONNX models into optimized secure C++ training code, enabling seamless integration into privacy-preserving ML pipelines—without requiring cryptographic knowledge.

Experimental results demonstrate the efficiency and practicality of our approach. On the Breast Cancer dataset, *HE-SecureNet* achieves a 5.2× speedup and 33% higher accuracy compared to ConcreteML (Zama) and TenSEAL (OpenMined). On the MNIST dataset, it reduces CNN training latency by 2× relative to Glyph (Lou et al., NeurIPS’20), and cuts communication overhead by up to 66× on MNIST and 42× on CIFAR-10 compared to MPC-based solutions.

## 1 Introduction

Today, machine learning (ML) services have become integral to critical domains such as healthcare, finance, biometrics, and environmental monitoring [9, 19, 21, 22, 24, 26, 27, 29, 31]. To enable widespread deployment of these services, cost-effective and energy-efficient embedded devices are commonly used to collect sensor data and perform local inference. However, these devices are constrained by limited computational power and energy capacity, making on-device training of complex ML models impractical [28].

*Machine Learning as a Service* (MLaaS) has emerged as a powerful solution, offering scalable cloud-based training through platforms such as Amazon AWS, Google Cloud, and Microsoft Azure [25].

These platforms allow users to easily upload their training data and obtain trained models via user-friendly interfaces [30, 48]. Yet, the reliance on external cloud infrastructure for training sensitive data (e.g., medical or financial records) has raised significant privacy concerns [12, 13, 37, 43]. In response, regulations such as the General Data Protection Regulation (GDPR) and California Consumer Privacy Act (CCPA) mandate strict data protection requirements [2, 5, 5, 49], compelling the development of private training mechanisms.

*Privacy-Preserving Machine Learning* (PPML) applies cryptographic tools to address these privacy challenges. Among the most prominent techniques are *Secure Multi-Party Computation* (MPC) [12, 26, 32, 39, 40, 43] and *Homomorphic Encryption* (HE) [9, 34, 41, 47]. MPC is particularly effective in collaborative training scenarios involving multiple data owners, as it enables joint model training without revealing private inputs. However, MPC-based protocols either require all participants to stay online throughout the training process or rely on multiple non-colluding servers—both of which are impractical for *client-server* settings with a single passive data provider [6, 23]. In contrast, HE is well-suited for client-server deployments where the client encrypts their data once, remains offline, and delegates all training computations to the server. Training occurs directly on ciphertexts, ensuring that sensitive data is never exposed, even during computation. This setup is particularly attractive in deployments where:

- The client has limited resources or intermittent connectivity;
- The server is untrusted and acts alone;
- Regulatory or trust boundaries prohibit ongoing interaction.

Despite these advantages, existing HE-based training systems face important limitations:

1. Model expressiveness: Current frameworks offer limited support for complex models such as Convolutional Neural Networks (CNNs) [7, 51].
2. Training flexibility: Multi-epoch training typically requires repeated client-server communication [7, 34, 41, 51].
3. Non-linear operations: Functions like MaxPooling remain difficult to approximate efficiently under encryption [7, 34, 41, 51].
4. Usability: Non-experts often struggle to adopt HE-based tools due to cryptographic complexity [34, 41].

**Contributions.** In this work, we introduce *HE-SecureNet*, a novel HE-based framework designed to overcome these limitations and make secure, accurate, and practical model training possible in client-server settings. *HE-SecureNet* builds on the PEGASUS library [35] and combines two state-of-the-art schemes: CKKS [17]

\*Corresponding author.

<sup>†</sup>★ Please cite the accepted version of this paper published at WPES’25 [46].

for arithmetic and FHEW [20] for binary operations. By leveraging hybrid scheme-switching, HE-SecureNet supports both linear and non-linear computations with high efficiency. Furthermore, it introduces a new ciphertext level management strategy that facilitates multi-epoch training without requiring modification of the entire model’s weights. HE-SecureNet also emphasizes usability by automatically transforming standard ML models into secure C++ training code, enabling developers to adopt privacy-preserving training without requiring cryptographic expertise. We summarize our main contributions as follows:

1. **Efficient Privacy-Preserving Model Training:** HE-SecureNet enables secure training over encrypted data while allowing client to remain offline after uploading. All computations occur on ciphertexts, ensuring end-to-end data confidentiality.
2. **Support for Complex Models and Functions:** HE-SecureNet supports logistic regression, multi-layer perceptrons, and CNNs. It includes efficient implementations of key non-linear functions such as ReLU, Softmax, and MaxPooling.
3. **Level Management for Multi-Epoch Training:** A novel strategy elevates the level of layer outputs instead of modifying model weights, eliminating level synchronization overhead in over 99% of ciphertext operations during multi-round training.
4. **Automated Code Transformation:** HE-SecureNet automatically converts model graphs in the ONNX format—used widely across ML ecosystems—into optimized, secure C++ training code. This greatly simplifies adoption for non-cryptographers.
5. **Enhanced Performance:** Our experiments show up to 5.2× faster training than ConcreteML [51] for logistic regression and 2× improvement over Glyph [34] for CNNs, with faster convergence.
6. **Reduced Communication Overhead:** Unlike MPC-based solutions, HE-SecureNet maintains constant communication regardless of batch size and achieves up to 66× less communication compared to existing MPC approaches [32].
7. **Open-Source Implementation:** Our framework is open-sourced at <https://encrypto.de/code/HE-SecureNet>.

## 2 Related Work

Table 1 compares HE-SecureNet with the most pertinent PPML frameworks. HE-SecureNet distinguishes itself by developing efficient and accurate training frameworks exclusively based on HE, accommodating a wide array of models, and enabling the computation of the MaxPooling function. Furthermore, it integrates with ML frameworks, enables developers without cryptographic expertise to use it, and is available as open source at <https://encrypto.de/code/HE-SecureNet>

**MPC-Based Frameworks.** MPC-based techniques, such as SecureML [39], QUOTIENT [3], and [32] enable collaborative model training by securely sharing data among multiple parties. However, these solutions require either multiple non-colluding servers or that the client and server stay online for multiple rounds of communication, resulting in high communication bandwidth requirements and several rounds of interaction.

**HE-Based Frameworks.** Although HE enables computations directly on encrypted data, all existing HE-based frameworks [7, 34, 41, 51, 52] still require client assistance to re-encrypt the model for multi-epoch training. TenSEAL [7] by OpenMined and ConcreteML [51] by Zama are effective for simple models like logistic regression, but have limited support for more complex models, such as neural networks. FHESGD [41] and HEPrune [52] lack support for non-linear layers, relying on approximations instead. Glyph [34] supports non-linear functions by combining the HE schemes BGV [11] and TFHE [18]. However, Glyph does not support the complex MaxPooling function and requires cryptographic expertise of its users. Furthermore, it relies on computationally expensive switching schemes, limiting its usability and efficiency. Note that although [33] employs the same PEGASUS library for PPML inference, HE-SecureNet broadens the scope to include PPML training.

**Hybrid Frameworks.** Hybrid frameworks like CryptoTrain [50] combine HE and MPC to optimize trade-offs between computational and communication overheads. It adopts the two-party setting for MPC protocols, and precompute secret shares to release the burden of online HE computations. However, it requires decryption and re-encryption for each layer, which increases complexity, like other HE-based works [7, 34, 41, 51].

## 3 Preliminaries

This section provides background on the core techniques used in HE-SecureNet. We begin by outlining Convolutional Neural Networks (CNNs), followed by an overview of the standard (plaintext) CNN training process. We then introduce Homomorphic Encryption (HE). For reference, Table 2 summarizes the key notations used throughout the paper.

### 3.1 CNN

CNNs are a class of artificial neural networks specifically designed to efficiently process spatially structured data through convolutional layers. We briefly summarize the key components used in CNNs next.

**Convolutional Layer.** Convolutional layers process spatial data using convolution operations. Given input feature maps  $\mathbf{D} \in \mathbb{R}^{w_D \times h_D \times c_D}$ , a convolutional layer computes output feature maps using a kernel tensor  $\mathbf{K} \in \mathbb{R}^{r \times r \times c_D \times c_K}$  and bias vector  $\mathbf{b} \in \mathbb{R}^{c_K}$ . For stride length  $s$ , the convolution is defined as:

$$(\mathbf{D} * \mathbf{K})_{i,j,k} = \mathbf{b}_k + \sum_{\gamma=1}^{c_D} \sum_{\alpha=1}^r \sum_{\beta=1}^r \mathbf{D}_{is+\alpha, js+\beta, \gamma} \cdot \mathbf{K}_{\alpha, \beta, \gamma, k}.$$

**Dense Layer.** A dense layer connects each input neuron to each output neuron. Given weights  $\mathbf{W} \in \mathbb{R}^{w_W \times h_W}$ , bias  $\mathbf{b} \in \mathbb{R}^{h_W}$ , and input feature  $\mathbf{d}$ , the output is computed as:

$$\mathbf{f} = \mathbf{W}\mathbf{d} + \mathbf{b}.$$

**Activation Function.** CNNs typically use non-linear activation functions such as ReLU and Softmax to enhance the model’s classification capability:

- **ReLU activation:** ReLU outputs the maximum value between zero and the input:

$$\text{ReLU}(x) = \max(0, x),$$

**Table 1: Comparison of Privacy-Preserving Training Frameworks.** LR: logistic regression, MLP: multi-layer perceptron, CNN: convolutional neural network.

| Name                            | Technique       | Model Support | Non-Interactive Training for Multi-Epoch | MaxPooling Support | PyTorch Support | Open Source |
|---------------------------------|-----------------|---------------|------------------------------------------|--------------------|-----------------|-------------|
| SecureML [39]                   | MPC             | LR            | ✗                                        | ✗                  | ✗               | ✗           |
| QUOTIENT [3]                    | MPC             | LR            | ✗                                        | ✓                  | ✗               | ✗           |
| Keller and Sun [32]             | MPC             | LR, MLP, CNN  | ✗                                        | ✓                  | ✓               | ✓           |
| CryptoTrain [50]                | Hybrid (HE+MPC) | LR            | ✗                                        | ✓                  | ✗               | ✗           |
| TenSEAL [7]                     | HE (BFV)        | LR            | ✗                                        | ✗                  | ✓               | ✓           |
| ConcreteML [51]                 | HE (TFHE)       | LR            | ✗                                        | ✗                  | ✓               | ✓           |
| FHESGD [41]                     | HE (BGV)        | MLP           | ✗                                        | ✗                  | ✗               | ✗           |
| HEPrune [52]                    | HE (CKKS)       | MLP           | ✗                                        | ✗                  | ✗               | ✓           |
| Glyph [34]                      | HE (BGV+TFHE)   | LR, MLP, CNN  | ✗                                        | ✗                  | ✗               | ✗           |
| <b>HE-SecureNet (this work)</b> | HE (CKKS+FHEW)  | LR, MLP, CNN  | ✓                                        | ✓                  | ✓               | ✓           |

**Table 2: A Summary of Notations**

| Notation                                  | Description                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------------------|
| $x, x^{(i)}$                              | Scalar and $i$ -th sample of scalar $x$                                                |
| $\mathbf{x}, x_i$                         | Vector and $i$ -th element of vector $\mathbf{x}$                                      |
| $\mathbf{X}$                              | Multi-dimensional tensor                                                               |
| $X_{i,j}, X_{i:k,j:l}$                    | Element $(i, j)$ and Elements from $(i, j)$ to $(i+k-1, j+l-1)$ in tensor $\mathbf{X}$ |
| $w_X, h_X, c_X$                           | Width, height, and channel of tensor $\mathbf{X}$                                      |
| $B, L, T, \eta$                           | Batch size, total layers, total classes, learning rate in a CNN model                  |
| $\mathbf{f}, \mathbf{d}$                  | Output from a linear layer, output from a non-linear layer                             |
| $\mathbf{W}^{(i)\{j\}}$                   | Weights of the $i$ -th layer in the $j$ -th epoch                                      |
| $\mathcal{F}(\cdot), \mathcal{SF}(\cdot)$ | Linear functions in the plaintext and encrypted domain                                 |
| $\mathcal{N}(\cdot), \mathcal{SN}(\cdot)$ | Non-linear functions in the plaintext and encrypted domain                             |
| $\mathcal{A}(\cdot), \mathcal{A}'(\cdot)$ | Activation function and its derivative                                                 |
| $\lambda, q, n$                           | Security parameter, ciphertext modulus, lattice dimension                              |
| $\Delta$                                  | Scaling factor                                                                         |
| $\mathbb{Z}_q, \mathbb{Z}_q[x]$           | Set of numbers modulo $q$ , polynomials with coefficients in $\mathbb{Z}_q$            |
| $R_{n,q}$                                 | Cyclotomic ring $\mathbb{Z}_q/(X^n + 1)$                                               |
| $\tilde{x}$                               | Ring element                                                                           |
| $[x]_R, [x]_L$                            | RLWE and LWE ciphertexts                                                               |
| $\oplus, \ominus, \otimes$                | Homomorphic addition, subtraction, and multiplication                                  |

and its derivative is defined as:

$$dReLU(x) = (x > 0) ? 1 : 0.$$

- *Softmax activation*: It is applied at the final layer to estimate probabilities across the  $T$  classes:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{T-1} e^{x_j}} \forall i \in [0, T-1].$$

*MaxPooling Function.* The  $r \times r$  MaxPooling function segments a feature map  $\mathbf{D} \in \mathbb{R}^{w_D \times h_D \times c_D}$  into  $r \times r$  blocks, selecting the maximum value within each block. The output is:

$$\mathbf{D}_{i,j,k}^M = \max(\mathbf{D}_{r(i-1)+1:r(i-1)+r, r(j-1)+1:r(j-1)+r, k}).$$

A tensor  $\mathbf{M}$ , with the same dimension as  $\mathbf{D}$ , is utilized to record the position of the maximum value within a pooling block. During backpropagation, gradients are propagated solely to the elements that yield the maximum value, while other positions do not contribute. Thus, the tensor  $\mathbf{M}$  can be defined as follows:

$$\mathbf{M}_{\alpha,\beta,k} = \begin{cases} 1 & \text{if } \mathbf{D}_{\alpha,\beta,k} = \mathbf{D}_{\lfloor \frac{\alpha}{r} \rfloor, \lfloor \frac{\beta}{r} \rfloor, k}^M \\ 0 & \text{otherwise.} \end{cases}$$

### 3.2 Plaintext CNN Model Training

The CNN model training typically consists of three phases: forward propagation, backpropagation, and weight updates. During each epoch, a batch containing  $B$  samples are randomly selected. Each sample is a set of features and labels, denoted as  $\mathbf{X}^{(i)}$  and  $\mathbf{y}^{(i)}$ , respectively. The training continues for multiple epochs until the model achieves convergence or meets predefined stopping criteria. We detail the training process for an  $L$ -layer CNN model below.

*Forward Propagation.* Each training sample is fed into the CNN layers from input to generate a predictive output  $\hat{\mathbf{y}}^{(i)}$ . The  $i$ -th layer output is computed as:

$$\mathbf{d}^{(i)} = \mathcal{A}^{(i)}(\mathbf{f}^{(i)}) = \mathcal{A}^{(i)}(\mathcal{F}^{(i)}(\mathbf{W}^{(i)}, \mathbf{d}^{(i-1)})).$$

*Backpropagation.* Backpropagation computes the gradients of the loss function with respect to each layer's weights. The cross-entropy loss, commonly used for classification tasks, is defined as:

$$\mathcal{E}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log(\hat{y}_i).$$

The gradient of the weights at the  $k$ -th layer,  $\mathbf{W}^{(k)}$ , is computed using the chain rule:

$$\nabla \mathbf{W}_{i,j}^{(k)} = \frac{\partial \mathcal{E}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(k)}} = \delta^{(k)} \cdot (\mathbf{d}^{(k-1)})^\top, \quad (1)$$

where

$$\delta^{(k)} = (\mathbf{W}^{(k+1)})^\top \delta^{(k+1)} \circ \mathcal{A}'(\mathbf{f}^{(k)}). \quad (2)$$

Backpropagation begins at the output layer  $L$ , where:

$$\delta^{(L)} = \hat{y} - y.$$

*Weight Updates.* The batch gradient is calculated by averaging gradients from each training sample in the batch and then scaled by the learning rate  $\eta$ . Weights are updated by moving in the opposite direction of the gradient. For the weights  $\mathbf{W}_{i,j}^{(k)}$ , the updated weights are defined as follows:

$$\mathbf{W}_{i,j}^{(k)\{e+1\}} = \mathbf{W}_{i,j}^{(k)\{e\}} - \frac{\eta}{B} \sum_{\alpha=0}^{B-1} \nabla \mathbf{W}_{i,j}^{(k)\{e\}}(\alpha).$$

### 3.3 Homomorphic Encryption

We primarily use two HE schemes: FHEW [20] and CKKS [17], which are based on the Learning with Errors (LWE) [45] and Ring Learning with Errors (RLWE) [36] problems, respectively. This section also covers the methods for converting between these two HE schemes and performing non-linear evaluations on LWE ciphertexts from PEGASUS [35]. For clarity, we use  $\underline{n}$  and  $n$  to denote two different power-of-two lattice dimensions, with  $\underline{q}$  and  $q$  as their respective ciphertext moduli. We assume that  $\underline{n} < n$  and  $\underline{q} < q$ . Additionally, the secret keys for these dimensions are denoted as  $\underline{s}$  and  $s$ .

*LWE-based HE (FHEW [20]).* The core components of an LWE-based HE scheme, as used in FHEW, are detailed as follows:

- **Key Generation:** Given a security parameter  $\lambda$ , the key generation function  $\mathcal{KL}^{n,q}(\lambda)$  produces a secret key  $\mathbf{s} \in \mathbb{Z}_q^n$  and a public key  $\mathbf{p} = (-\mathbf{a}^\top \mathbf{s} + e, \mathbf{a})$ , where  $\mathbf{a}$  is a vector with each element randomly selected from  $\mathbb{Z}_q$ , and the error term  $e$  is drawn from a designated error distribution  $\chi_{err}$ .
- **Encryption:** The encryption function  $\mathcal{EL}_p^{n,q}(m)$  encrypts a message  $m \in \mathbb{Z}_q$  as follows:

$$[m]_L = \mathcal{EL}_p^{n,q}(m) = (b, \mathbf{a}) = (m + e - \mathbf{a}^\top \mathbf{s}, \mathbf{a}).$$

- **Decryption:** The decryption function  $\mathcal{DL}_s^{n,q}([m]_L)$  recovers the message  $m$  by:

$$m \approx \mathcal{DL}_s^{n,q}([m]_L) = m + e.$$

- **Homomorphic Addition:** Consider two LWE ciphertexts that encrypt messages  $m_0$  and  $m_1$  respectively:  $[x^{(0)}]_L = (b^{(0)}, \mathbf{a}^{(0)})$  and  $[x^{(1)}]_L = (b^{(1)}, \mathbf{a}^{(1)})$ . The homomorphic addition operation  $\oplus$  is performed as follows:

$$\begin{aligned} [x^{(0)}]_L \oplus [x^{(1)}]_L &= (b^{(0)} + b^{(1)}, \mathbf{a}^{(0)} + \mathbf{a}^{(1)}) \\ &= \mathcal{EL}_s^{n,q}(m^{(0)} + m^{(1)}). \end{aligned}$$

*RLWE-based HE (CKKS [17]).* An RLWE-based HE scheme, such as CKKS, operates over polynomial rings and supports approximate arithmetic on real or complex numbers. Its main building blocks are summarized below:

- **Key Generation:** Given the security parameter  $\lambda$ , the RLWE-based key generation function  $\mathcal{KR}^{n,q}(\lambda)$  generates a secret key  $\tilde{s} \in R_n$  and a public key  $\tilde{\mathbf{p}} = (-\tilde{a} \cdot \tilde{s} + \tilde{e}, \tilde{a})$ . For this,  $\tilde{a}$  is chosen uniformly at random, and  $\tilde{e}$  is sampled from the error distribution  $\chi_{err}$ .

- **Encode:** To handle floating-point numbers typically used in ML models, an encoding function  $\mathcal{ECD}(\Delta, \mathbf{m})$  is used to convert an  $\mathbf{m} \in \mathbb{R}^n$  into a ring element  $\tilde{m} \in R_{n,q}$ , scaled by  $\Delta$ .

- **Encryption:** The RLWE encryption function, denoted as  $\mathcal{ER}_p^{n,q}(\tilde{m})$ , encrypts a message  $\tilde{m} \in R_{n,q}$  by computing:

$$[\tilde{m}]_R = \mathcal{ER}_p^{n,q}(\tilde{m}) = (\tilde{b}, \tilde{\mathbf{a}}) = (\tilde{m} - \tilde{a} \cdot \tilde{s} + \tilde{e}, \tilde{a}).$$

- **Decryption:** The decryption function  $\mathcal{DR}_s^{n,q}([\tilde{m}]_R)$  recovers the message approximately as follows:

$$\tilde{m} \approx \mathcal{DR}_s^{n,q}([\tilde{m}]_R) = \tilde{m} + e.$$

- **Homomorphic Operations:** For two RLWE ciphertexts,  $[\tilde{m}^{(0)}]_R = (\tilde{b}^{(0)}, \tilde{a}^{(0)})$  and  $[\tilde{m}^{(1)}]_R = (\tilde{b}^{(1)}, \tilde{a}^{(1)})$ , encrypting  $\tilde{m}^{(0)} = \mathcal{ECD}(\Delta, \mathbf{m}^{(0)})$  and  $\tilde{m}^{(1)} = \mathcal{ECD}(\Delta, \mathbf{m}^{(1)})$  respectively, homomorphic addition is performed as:

$$\begin{aligned} [\tilde{m}^{(0)}]_R \oplus [\tilde{m}^{(1)}]_R &= (\tilde{b}^{(0)} + \tilde{b}^{(1)}, \tilde{a}^{(0)} + \tilde{a}^{(1)}) \\ &= \mathcal{ER}_p^{n,q}(\tilde{m}^{(0)} + \tilde{m}^{(1)}). \end{aligned}$$

Similarly, homomorphic multiplication is:

$$\begin{aligned} [\tilde{m}^{(0)}]_R \otimes [\tilde{m}^{(1)}]_R &= \mathcal{ER}_p^{n,q}(\tilde{m}^{(0)} \times \tilde{m}^{(1)}) \\ &= (\tilde{b}^{(0)} \cdot \tilde{b}^{(1)}, \tilde{b}^{(0)} \cdot \tilde{a}^{(1)} + \tilde{b}^{(1)} \cdot \tilde{a}^{(0)}, \tilde{a}^{(0)} \cdot \tilde{a}^{(1)}). \end{aligned}$$

- **Batching:** The RLWE-based HE supports batching, enabling simultaneous computations on multiple ciphertext elements. For example, a single HE addition results in the  $i$ -th slot of  $[\tilde{m}^{(2)}]_R$  being the encryption of  $\mathbf{m}_i^{(0)} + \mathbf{m}_i^{(1)}$ .
- **Leveled Ciphertext Modulus:** An  $L$ -level ciphertext modulus  $q$  consists of a product of  $L$  co-prime moduli  $p_i$ , i.e.,  $q_L = \prod_{i=1}^L p_i$ . Each homomorphic multiplication reduces the ciphertext level to control noise growth. When the level approaches zero, bootstrapping [16] is applied to refresh the ciphertext by reducing noise and restoring it to a higher level, allowing further computations. It is important to note that a HE operation with a lower level is faster, as it computes over a smaller ciphertext modulus.

- **Rotation:** The rotation function  $\mathcal{ROT}([\tilde{m}]_R, i)$  cyclically shifts the vector  $\mathbf{m}$  encrypted in  $[\tilde{m}]_R$  by  $i$  positions to the left without requiring decryption. Specifically, the operation is defined as:

$$\begin{aligned} [\tilde{m}']_R &= \mathcal{ER}_p^{n,q}(\mathcal{ECD}(\Delta, \{\mathbf{m}_i, \mathbf{m}_{i+1}, \dots, \mathbf{m}_{i-1}\})) \\ &= \mathcal{ROT}([\tilde{m}]_R, i). \end{aligned}$$

### 3.4 Homomorphic Scheme Conversions in PEGASUS [35]

The conversion from RLWE (CKKS) to LWE (FHEW), denoted as  $\mathcal{R2L}$ , transforms an RLWE ciphertext into  $n$  LWE ciphertexts:

$$\mathcal{R2L}([\tilde{m}]_R) = [\mathbf{m}]_L,$$

where

$$[\mathbf{m}_i]_L = \mathcal{EL}_s^{n,q}(\Delta \mathbf{m}_i) \forall i \in [0, n-1].$$

In contrast, the function  $\mathcal{L2R}$  transforms multiple LWE ciphertexts to an RLWE ciphertext using a repacking key  $rk \in$



$$\mathcal{ER}_{\mathbf{p}}^{n,q'}(\mathcal{ECD}(\Delta, \mathbf{s})):$$

$$\mathcal{L}2\mathcal{R}_{rk}(\{[\mathbf{m}_0]_L, \dots, [\mathbf{m}_{n-1}]_L\}) = [\tilde{\mathbf{m}}']_R \in \mathcal{ER}_{\mathbf{p}}^{n,q''}(\tilde{\mathbf{m}}).$$

Upon repacking, the ciphertext modulus of  $[\tilde{\mathbf{m}}']_R$  is refilled and could be utilized for HE operations. The quantity of refilled levels can be determined upon parameter selections.

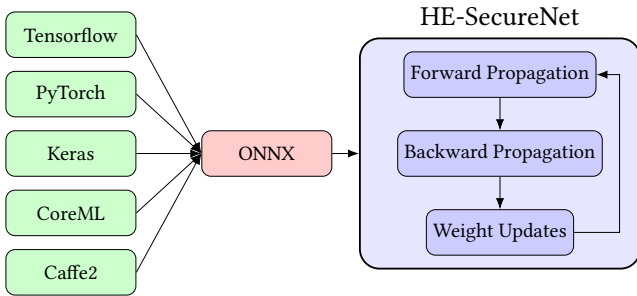
*Programmable Bootstrapping (PBS).* Consider an LWE ciphertext  $[x]_L = \mathcal{EL}_{\mathbf{s}}^{n,q}(\Delta m)$ , where  $m \in \mathbb{R}$ . The programmable bootstrapping on LWE ciphertexts can reduce ciphertext noise while simultaneously enabling non-linear evaluations. Utilizing a bootstrapping key  $bk$  and a lookup table (LUT) for a non-linear function  $\mathcal{N}$ , defined as  $T_{\mathcal{N}}(m) : \mathbb{R} \rightarrow \mathbb{R}$ , the resulting bootstrapped ciphertext  $[x']_L$  is computed as:

$$[x']_L = \mathcal{PBS}_{bk, \mathcal{N}}([x]_L) = \mathcal{EL}_{\mathbf{s}}^{n,q}(\Delta T_{\mathcal{N}}(m) + e),$$

where  $e$  represents a constant, input-independent error.

## 4 The HE-SecureNet Framework

Figure 1 presents an overview of our HE-SecureNet framework. The framework begins with an input model graph in the Open Neural Network Exchange (ONNX) [38] format, a versatile and open-source model format that can be generated from various machine learning toolkits such as TensorFlow [1], PyTorch [42], and Scikit-learn [44]. This compatibility allows developers to continue using familiar ML tools to construct their models. HE-SecureNet then automatically generates the PPML code from the ONNX model graph provided. This allows developers who are not experts in PPML to easily build secure ML applications.



**Figure 1: HE-SecureNet: Framework Overview.** HE-SecureNet is a system that combines existing ML frameworks (green blocks) with PEGASUS [35]-based private training methods (blue blocks) to enable secure training.

HE-SecureNet facilitates efficient private training, by utilizing the PEGASUS library [35], which enables seamless switching between CKKS and FHEW ciphertexts. HE-SecureNet can efficiently evaluate linear functions with batching using CKKS and accurately evaluate complex non-linear functions, such as ReLU, Softmax, and MaxPooling, using programmable bootstrapping with FHEW. To facilitate efficient non-interactive multi-epoch training, HE-SecureNet selectively refills the levels of each layer’s output instead of resetting the levels of all model weights.

Compared to Chimera [10], PEGASUS [35] showed a 64× speed improvement for each programmable bootstrapping and over 174×

improvement for scheme conversions. Although [4, 8, 53] presents updated scheme conversion algorithms, [8, 53] exhaust all multiplicative levels after LWE ciphertexts are reverted to the RLWE ciphertext, rendering them unsuitable for model training due to the necessity of costly bootstrapping before each linear function. For [4], the implementation is not released. Thus, we select PEGASUS [35] as HE backend.

### 4.1 HE-based Model Training Workflow

In the HE-based PPML training workflow, the client homomorphically encrypts their private data and sends it to the server. Once the encrypted data is uploaded, the client can go offline. The server trains the model under encryption, ensuring no decryption occurs at any stage. When the training process is completed, the client reconnects to the server to retrieve the model weights. The entire workflow is divided into three main steps, which are detailed as follows:

#### 1. Encrypted Training Data Preparation

- **Client:** Encrypts a batch of training samples using her public key and sends the encrypted data to the server. This step ensures that sensitive data remains confidential, as the server can process data without gaining access to the unencrypted plaintexts.

#### 2. Secure Model Training

- **Server:** Conducts training on the encrypted data across multiple epochs. The server performs computations needed for training without ever decrypting the data, preserving the privacy of the input and informative values.

#### 3. Encrypted Model Release

- **Server:** Upon finishing the training, the server returns the encrypted model to the client. This approach ensures the confidentiality of the trained model’s details.
- **Client:** Decrypts the received model with her private key, gaining access to the trained model.

### 4.2 Threat Model and Security of HE-SecureNet

In our framework, we assume a semi-honest (honest-but-curious) server, meaning that the server adheres to the protocol specifications but may attempt to infer sensitive information from the encrypted training data. Under this assumption, the integrity of model weights is preserved, while the confidentiality of both the model and training data is protected by encryption throughout training. Only the model structure represents the sole information accessible to the server.

The security of HE-SecureNet is fundamentally connected to that of the underlying Fully Homomorphic Encryption (FHE) schemes, specifically CKKS [17] and FHEW [20], both of which provide security against chosen plaintext attacks, thereby ensuring the security of the semi-honest model. Additionally, HE-SecureNet incorporates protocols for scheme conversions and programmable bootstrapping from PEGASUS [35]. Assuming the circular security of HE [11], switching between different ciphertext formats preserves the overall semi-honest security as well.

### 4.3 Key Processes and Functions of HE-SecureNet

**Encrypted Training Data Preparation.** The client encrypts  $B$  training samples and labels into RLWE ciphertexts as follows:

- **Encrypted Training Samples:** A three-dimensional (3-D) tensor  $[X]_R$  is constructed, with dimensions matching the shape of the training sample  $X$ . Each element  $[X_{i,j,k}]_R$  encrypts a total of  $B$  samples, which implies that:

$$[X_{i,j,k}]_R = \mathcal{ER}_{\mathbf{p}}^{n,q}(\mathcal{ECD}(\Delta, \{X_{i,j,k}^{(0)}, \dots, X_{i,j,k}^{(B-1)}\}))$$

$$\forall i \in [0, w_X - 1], j \in [0, h_X - 1], \text{ and } k \in [0, c_X - 1].$$

- **Encrypted Training Labels:** An encrypted label vector  $[y]_R$  is generated in a manner analogous to that of the training samples.  $[y_{i,j}]_R$  denotes the labels corresponding to the  $i$ -th class in the one-hot encoded ground truth of the  $j$ -th training sample.

**Secure Linear Functions.** To securely compute linear functions—such as convolutions, dense layers, gradient computations in backpropagation, and model weight updates—we convert plaintext operations of a linear function  $\mathcal{F}$  into their HE counterparts. Consider  $B$  real-numbered feature maps denoted as  $\{d^{(0)}, d^{(1)}, \dots, d^{(B-1)}\}$ . In the non-encrypted domain, the function  $\mathcal{F}$  will be invoked  $B$  times to obtain all the feature map outputs:  $f^{(i)} = \mathcal{F}(d^{(i)})$ .

The corresponding encrypted linear function is denoted as  $[f]_R = \mathcal{SF}([d]_R)$ , where  $[d_i]_R$  encrypts the  $i$ -th components from each feature map  $d_i^{(0)}$  through  $d_i^{(B-1)}$ . With the CKKS batching, the function  $\mathcal{SF}$  needs to be executed only once, resulting in  $[f_i]_R$  which comprises the  $i$ -th components of the outputs  $\{f_i^{(0)}, f_i^{(1)}, \dots, f_i^{(B-1)}\}$ . Because batch samples of an element are encrypted within the same ciphertext, HE-SecureNet can efficiently reshape or flatten tensors as procedures to unencrypted tensors without shuffling the batch orders.

**Secure Non-linear Functions.** Using the PEGASUS library [35], non-linear functions can be implemented securely with multiple FHEW operations. A secure non-linear function processes one or more CKKS ciphertexts as inputs and generates one or more CKKS ciphertexts as outputs. In the following, we describe the implementation of secure versions of commonly used non-linear functions, including ReLU, Softmax, and MaxPooling.

**Secure ReLU & dReLU.** The secure ReLU function processes an input CKKS ciphertext  $[a]_R$  and produces an output CKKS ciphertext  $[b]_R$ . This function is implemented efficiently using just one PBS function:

1. The server converts the CKKS ciphertext  $[a]_R$  into  $n$  FHEW ciphertexts:  $[a]_L = R2L([a]_R)$ .
2. Each FHEW ciphertext  $[a_i]_L$  is then processed through a PBS to apply the ReLU function, resulting in:  $[b_i]_L = \mathcal{PBS}_{bk,ReLU}([a_i]_L) \forall i \in [0, n-1]$ .
3. Finally, the server combines these results back into a single CKKS ciphertext:  $[b]_R = \mathcal{L2R}_{rk}([b]_L)$ .

Similarly, the derivative of ReLU (dReLU) is computed using the same procedure but with a different LUT, substituting  $\mathcal{PBS}_{bk,ReLU}(\cdot)$  with  $\mathcal{PBS}_{bk,dReLU}(\cdot)$  in the process.

**Secure Softmax.** Following the evaluation of the last neural network layer  $\mathcal{F}^{(L)}$ , we obtain the encrypted outputs as  $[f^{(L)}]_R$ . To securely compute the Softmax function, the encrypted outputs  $[f^{(L)}]_R$  are converted into the FHEW scheme, forming a matrix  $[F]_L$ . Each element  $[F_{i,j}]_L$  corresponds to the  $i$ -th feature of the encrypted outputs for the  $j$ -th training sample. Each column of  $[F]_L$  is then processed individually by the secure Softmax operation.

Since the PEGASUS library does not support multiplication of two FHEW ciphertexts, we modify the Softmax computation by rewriting it in terms of logarithmic differences:

$$\begin{aligned} \text{Softmax}(\mathbf{x}_i) &= e^{\log\left(\frac{e^{x_i}}{\sum_j e^{x_j}}\right)} \\ &= e^{\log(e^{x_i}) - \log(\sum_j e^{x_j})} \\ &= e^{x_i - \log(\sum_j e^{x_j})}. \end{aligned} \quad (3)$$

The implementation of this modified Softmax function is detailed in Algorithm 1.

---

#### Algorithm 1 Secure Softmax.

---

**Require:**  $T$ -element ciphertext  $[x]_R$ , where  $[x_i]_R$  encrypts a batch of  $B$  values:  $\{x_i^{(0)}, x_i^{(1)}, \dots, x_i^{(B-1)}\}$ . The bootstrapping key  $bk$  and the repacking key  $rk$ .

**Ensure:**  $T$ -element ciphertext  $[y]_R$ , where  $[y_i]_R$  encrypts a batch of  $B$  values:  $\{y_i^{(0)}, y_i^{(1)}, \dots, y_i^{(B-1)}\}$ , computed as  $y_i^{(j)} = \text{Softmax}(x_i^{(j)})$  for inputs  $\{x_0^{(j)}, x_1^{(j)}, \dots, x_{T-1}^{(j)}\}$ .

- 1: Convert ciphertexts from CKKS to FHEW:  $[A_i]_L = R2L([x_i]_R) \forall i \in [0, T-1]$
- 2: **for**  $0 \leq j < B$  **do**
- 3:   Initialize accumulator ciphertext:  $[sum]_L = [0]_L$
- 4:   **for**  $0 \leq i < T$  **do**
- 5:     Compute encrypted exponent and accumulate:  $[sum]_L = [sum]_L \oplus \mathcal{PBS}_{bk,Exp}([A_{i,j}]_L)$
- 6:   **end for**
- 7:   Compute logarithm of sum:  $[log\_sum]_L = \mathcal{PBS}_{bk,Log}([sum]_L)$
- 8:   **for**  $0 \leq i < T$  **do**
- 9:     Compute Softmax outputs:  $[A_{i,j}]_L = \mathcal{PBS}_{bk,Exp}([A_{i,j}]_L \ominus [log\_sum]_L)$
- 10:   **end for**
- 11: **end for**
- 12: Convert ciphertexts from FHEW back to CKKS:  $[y_i]_R = \mathcal{L2R}_{rk}([A_i]_L) \forall i \in [0, T-1]$

---

**Secure MaxPooling.** To securely compute the maximum of two elements  $m^{(0)}$  and  $m^{(1)}$ , we use the following formula:

$$\max(m^{(0)}, m^{(1)}) = 0.5(m^{(0)} + m^{(1)}) + 0.5|m^{(0)} - m^{(1)}|.$$

We implement this securely by leveraging two LUTs:

$$\text{Half}(x) = 0.5x, \quad \text{HalfAbs}(x) = 0.5|x|.$$

For brevity, we illustrate the secure  $r \times r$  MaxPooling function over the  $r \times r$  CKKS matrix  $[\mathbf{D}]_R$ . HE-SecureNet first transforms  $[\mathbf{D}]_R$  into a 3-D  $r \times r \times B$  tensor  $[\mathbf{D}]_L$ . The MaxPooling operation extracts  $B$  maximum values  $[\mathbf{m}]_L$  from each slice  $[\mathbf{D}_{:, :, k}]_L$ , and these  $[\mathbf{m}]_L$  are aggregated into a ciphertext representing the MaxPooling output for that region, denoted as  $[\tilde{\mathbf{m}}]_R$ .

To facilitate backpropagation, an additional LUT called INN identifies the positions of these maximum values within each slice:

$$\text{INN}([\mathbf{M}_{\alpha, \beta, k}]_L) = \begin{cases} 1 & \text{if } [\mathbf{D}_{\alpha, \beta, k}]_L - [\mathbf{m}_k]_L \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

The LUT indicates positions containing the maximum with a value of 1 and others with a value of 0. The 3-D binary position indicators  $[\mathbf{M}]_L$  are then encoded into 2-D CKKS ciphertexts  $[\mathbf{M}]_R$ .

In conclusion, the detailed procedure for secure  $r \times r$  MaxPooling is presented in Algorithm 2. Table 3 summarizes the number of operations for non-linear functions in HE-SecureNet.

---

**Algorithm 2** Secure  $r \times r$  MaxPooling.

---

**Require:** Encrypted  $r \times r$  matrix  $[\mathbf{D}]_R$  with  $B$  feature maps  $\{\mathbf{U}^{(0)}, \mathbf{U}^{(1)}, \dots, \mathbf{U}^{(B-1)}\}$ . The bootstrapping key  $bk$  and the repacking key  $rk$ .

**Ensure:** Encrypted pooled output  $[\tilde{\mathbf{m}}]_R$ , where  $\tilde{\mathbf{m}} = \mathcal{ECD}(\Delta, \mathbf{m})$  and  $\mathbf{m}_\alpha = \max(\mathbf{U}^{(\alpha)}) \forall \alpha \in [0, B-1]$ .

**Ensure:** Encrypted  $r \times r$  indicator matrix  $[\mathbf{M}]_R$ , where  $[\mathbf{M}_{i,j}]_R$  encrypts indicators  $\mathbf{n}_\alpha = \text{INN}(\mathbf{U}_{i,j}^{(\alpha)} - \mathbf{m}_\alpha)$  with  $\alpha \in [0, B-1]$ .

- 1: Convert to FHEW ciphertexts: Create an  $r \times r \times B$  tensor  $[\mathbf{D}]_L$ , where  $[\mathbf{D}_{i,j,:}]_L = \mathcal{R2L}([\mathbf{D}_{i,j}]_R)$ .
  - 2: Initialize  $B$ -length output vector  $[\mathbf{m}]_L$  to zeros.
  - 3: Initialize  $r \times r \times B$  indicator tensor  $[\mathbf{M}]_L$  to zeros.
  - 4: **for**  $0 \leq k < B$  **do**
  - 5:   Vectorize pooling region:  $[\mathbf{v}]_L = \text{vec}([\mathbf{D}_{0:r,0:r,k}]_L)$
  - 6:   Initialize maximum:  $[\mathbf{m}_k]_L = [\mathbf{v}_0]_L$
  - 7:   **for**  $1 \leq \alpha < r^2$  **do** ▷ Compute maximum
  - 8:      $[\mathbf{m}^{(0)}]_L = \mathcal{PBS}_{bk, Half}([\mathbf{m}_k]_L \oplus [\mathbf{v}_\alpha]_L)$
  - 9:      $[\mathbf{m}^{(1)}]_L = \mathcal{PBS}_{bk, HalfAbs}([\mathbf{m}_k]_L \ominus [\mathbf{v}_\alpha]_L)$
  - 10:    Update maximum:  $[\mathbf{m}_k]_L = [\mathbf{m}^{(0)}]_L \oplus [\mathbf{m}^{(1)}]_L$
  - 11:   **end for**
  - 12:   **for**  $0 \leq i < r$  **and**  $0 \leq j < r$  **do**
  - 13:     Compute position indicators:  $[\mathbf{M}_{i,j,k}]_L = \mathcal{PBS}_{bk, INN}([\mathbf{D}_{i,j,k}]_L \ominus [\mathbf{m}_k]_L)$
  - 14:   **end for**
  - 15: **end for**
  - 16: Convert outputs to CKKS:  $[\tilde{\mathbf{m}}]_R = \mathcal{L2R}_{rk}([\mathbf{m}]_L)$
  - 17: Convert indicators to CKKS:  $[\mathbf{M}_{i,j}]_R = \mathcal{L2R}_{rk}([\mathbf{M}_{i,j,:}]_L)$
- 

**Secure Weight Updates Function.** Due to the CKKS batching capability, each gradient for layer  $k$ , denoted as  $[\nabla \mathbf{W}_{i,j}^{(k)}]_R$ , encrypts gradients of  $B$  samples. We securely compute the batch-average gradient in two steps:

1. Sum all gradients by cyclically rotating  $[\nabla \mathbf{W}_{i,j}^{(k)}]_R$  ciphertexts  $\log_2 B$  times. In each rotation, the ciphertext is added to itself shifted by  $2^\alpha$  slots, with  $\alpha$  starting at  $\log_2 B - 1$  and decreasing by one each iteration.

**Table 3: Complexity analysis of non-linear evaluations in HE-SecureNet.**

| Function   | #HE Add.     | #R2L  | #PBS          | #L2R      |
|------------|--------------|-------|---------------|-----------|
| ReLU       | 0            | 1     | $B$           | 1         |
| Softmax    | $2BT$        | $T$   | $2BT + B$     | $T$       |
| MaxPooling | $B(r^2 - 1)$ | $r^2$ | $3B(r^2 - 2)$ | $r^2 + 1$ |

2. Multiply the resulting ciphertext by  $\eta/B$  to compute the average gradient and store it in the first slot.

For multi-epoch training, this average gradient must be uniformly replicated across all  $B$  slots. To achieve this, we perform:

1. Multiplication of  $[\nabla \mathbf{W}_{i,j}^{(k)}]_R$  by a filter ciphertext  $[f]_R$ , where only the first slot is set to one and all other slots are zero. This isolates the batch gradient.
2. Cyclical rotations of the filtered ciphertext  $[g]_R$  for  $\log_2 B$  iterations, progressively replicating the gradient to all slots, starting from  $\alpha = 0$  and increasing incrementally.

Finally, the uniformly distributed gradient is used to update the model weights by subtracting it from the current weights. The complete secure weight-update procedure is detailed in Algorithm 3.

---

**Algorithm 3** Secure Weight Updates.

---

**Require:** Encrypted weight matrix  $[\mathbf{W}]_R^{\{e\}}$  at epoch  $e$ , encrypted gradient matrix  $[\nabla \mathbf{W}]_R$ , batch size  $B$ , and learning rate  $\eta$ .

**Ensure:** Updated encrypted weight matrix  $[\mathbf{W}]_R^{\{e+1\}}$ .

- 1: **for**  $0 \leq i < w_W$  **and**  $0 \leq j < h_W$  **do**
  - 2:   // Sum gradients across all  $B$  slots
  - 3:   **for**  $\alpha = 0, \dots, \log_2(B) - 1$  **do**
  - 4:      $[\nabla \mathbf{W}_{i,j}]_R = [\nabla \mathbf{W}_{i,j}]_R \oplus \mathcal{ROT}([\nabla \mathbf{W}_{i,j}]_R, 2^\alpha)$
  - 5:   **end for**
  - 6:   // Isolate sum in the first slot
  - 7:    $[g]_R = [\nabla \mathbf{W}_{i,j}]_R \otimes [f]_R$  ▷  $[f]_R$ : filter ciphertext (first slot=1, others=0)
  - 8:   // Replicate sum uniformly across all slots
  - 9:   **for**  $\alpha = 0, \dots, \log_2(B) - 1$  **do**
  - 10:      $[g]_R = [g]_R \oplus \mathcal{ROT}([\nabla \mathbf{W}_{i,j}]_R, -2^\alpha)$
  - 11:   **end for**
  - 12:   // Compute average gradient and update weights
  - 13:    $[\mathbf{W}_{i,j}]_R^{\{e+1\}} = [\mathbf{W}_{i,j}]_R^{\{e\}} \ominus (\eta/B) \otimes [g]_R$
  - 14: **end for**
- 

**Encrypted Model Release.** To minimize communication overhead, the server reduces the ciphertext modulus of the encrypted model weights to the minimal secure level before releasing them. Thus, the client receives the model weights  $[\mathbf{W}_{i,j}^{(k)}]_R$  encrypted within the ring  $R_{n,q_0}^2$ .

#### 4.4 Ciphertext Level Management

Ciphertext level management is a critical issue for CKKS computations, as it is essential to ensure that the two ciphertexts involved possess the same and sufficient levels to guarantee the correctness

of the results. To enable multi-epoch training, ConcreteML [51], FHESGD [41], Glyph [34], HEPrune [52], and TenSEAL [7] elevate model weights to a higher CKKS level via client-assisted re-encryption, resulting in substantial communication overhead and undermining the non-interactive nature of HE-based training. Although ConcreteML [51] mentions level resetting of model weights through bootstrapping as a potential solution to preserve non-interactive training in HE-based methods, their approach is prohibitively expensive when managing a large number of parameters.

To mitigate extensive level adjustments on model weights, HE-SecureNet instead elevates the level of each layer’s output during forward and backward propagations (i.e.,  $[f]_R$  and  $[\delta]_R$ ) while maintaining the model weights at a constant level. Thereby, our approach significantly reduces the computational costs between training iterations. To further improve performance, HE-SecureNet could discard unnecessary levels of the ciphertexts before linear layers. Our level management works as follows:

1. The level adjustment is achieved by leveraging the property that the  $\mathcal{L}2R$  function replenishes the ciphertext level. Because both calculations of  $[d]_R = [\mathcal{A}(f)]_R$  and  $[\mathcal{A}'(f)]_R$  relate to PBS, they remain at high levels after generated. Although the calculation of  $[\delta]_R$  does not involve PBS, its level can still be replenished through a continuous workflow of  $\mathcal{R}2L$  and  $\mathcal{L}2R$  functions after generation.
2. Since the computation for backpropagation requires two levels (as indicated by Equation 2), the level of the weights must be at least two levels higher than the minimum level permissible for scheme conversions to ensure that the output of backpropagation can still be repacked.
3. Since the weight updates function takes one level (shown in Equation 1) for calculating  $[\nabla W]_R$ , the inputs of weight updates ( $[\delta]_R$  and  $[d]_R$ ) must be one level higher than that of the weights.
4. In accordance with the above three principles, the repacked ciphertexts ( $[d]_R$ ,  $[\delta]_R$ , and  $[\mathcal{A}'(f)]_R$ ) should be three levels higher than the minimum requirement. To achieve the best computing performance, inputs to linear functions will be reduced to the minimum executable levels in advance to accelerate the process.

Figure 2 illustrates our level management strategy. Given that the minimum level for the scheme conversion is  $l$ , the model weights will consistently be at level  $l + 2$ , while the refilled ciphertexts will be at level  $l + 3$ . Since the MaxPooling function is executed with FHEW ciphertexts, it does not affect the highest level required of the ciphertexts.

#### 4.5 C++ Code Generator for Private Training

HE-SecureNet employs the ONNX model graph to facilitate the integration of private model training with contemporary ML tools, including TensorFlow [1], PyTorch [42], and Scikit-learn [44]. The C++ code generated automatically utilizes the PEGASUS [35] library for HE operations, with sample code presented in Listing 1.

**Code Generation Process.** The generated code consists of three phases: forward propagation, backward propagation, and model weight updates. Forward propagation and weight updates process the model graph sequentially, starting from the input layer, while

backward propagation traverses the graph in reverse, computing gradients for each layer. The framework currently supports all essential neural network operations, including convolutional, dense, and max-pooling layers, as well as activation functions such as ReLU and Softmax.

**Code Example.** Listing 1 shows an example of automatically generated privacy-preserving training code for a Multi-Layer Perceptron (MLP). The example MLP consists of three dense layers: two hidden layers with 128 ReLU-activated units each, followed by an output layer of 10 units with Softmax activation. The training process includes forward propagation for output computation, backward propagation for gradient calculation, and weight updates to optimize the model over multiple epochs. The code is generated with negligible latency and is immediately ready for private training, demonstrating HE-SecureNet’s capability to assist non-cryptographic experts in end-to-end privacy-preserving neural network training.

## 5 Evaluation

### 5.1 Experimental Methodology

**Cryptosystem Configuration.** Table 4 summarizes the cryptographic parameters employed for the various models in HE-SecureNet. Since HE-SecureNet utilizes the PEGASUS library [35], most parameters are derived from PEGASUS and achieve a security level of 119 bits based on the RLWE hardness assumption. The Logistic Regression model can be executed with a lower ciphertext modulus in the RLWE scheme, as it does not have any multiplicative level during backpropagation. In contrast, the MLP, CNN, and LeNet-5 models require a total of three levels for the training process; thus, the ciphertext modulus is slightly higher.

**Table 4: Cryptographic Parameters for HE-SecureNet. Here,  $n$  represents the lattice dimension, and  $q$  denotes the ciphertext modulus.**

| Scheme | Parameters                   | Models                                 |
|--------|------------------------------|----------------------------------------|
| LWE    | $n = 1,024, \log_2 q = 46$   | Logistic Regression, MLP, CNN, LeNet-5 |
| RLWE   | $n = 65,536, \log_2 q = 749$ | Logistic Regression                    |
| RLWE   | $n = 65,536, \log_2 q = 795$ | MLP, CNN, LeNet-5                      |

**Experimental Setup.** The experiments were conducted on a server equipped with an AMD Ryzen Threadripper 3990X CPU with 512 GB of RAM.

**Datasets.** We evaluated HE-SecureNet on the following datasets:

- **Breast Cancer Dataset:** Comprising 569 samples with 30 numerical features each, this dataset was used to train a logistic regression model for classifying tumors as benign or malignant.
- **MNIST Dataset:** A benchmark dataset of handwritten digits, consisting of 60,000 training samples and 10,000 test samples, each represented as  $28 \times 28$  grayscale images.
- **CIFAR-10 Dataset:** A widely used dataset for image classification, consisting of 60,000 color images of size  $32 \times 32$  pixels, categorized into 10 distinct classes.



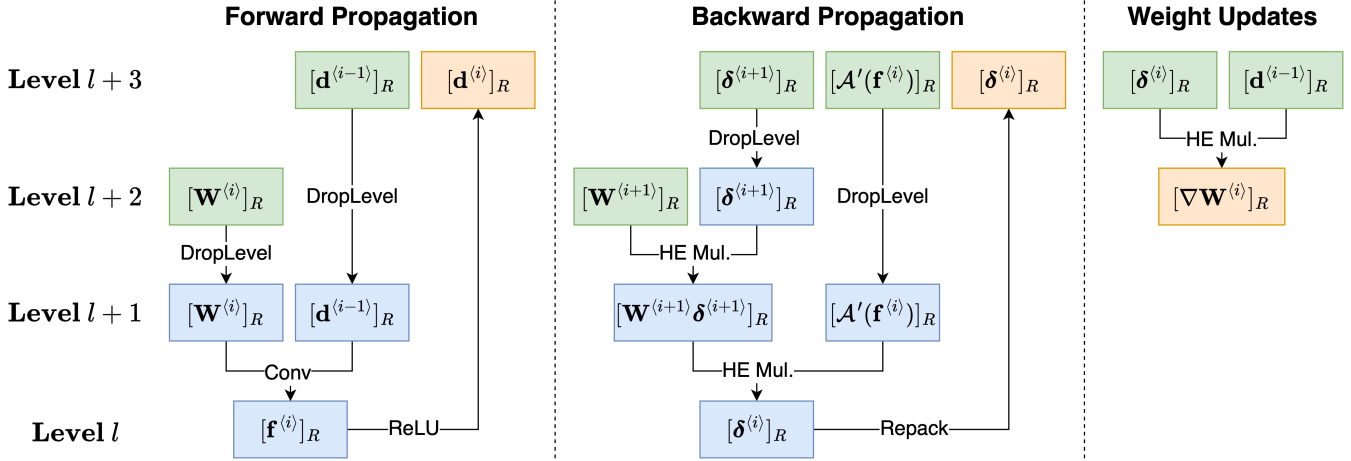


Figure 2: The level applied to calculate the  $i$ -th layer output during training while utilizing the repacking-efficient level management in HE-SecureNet. Green blocks are inputs, orange blocks are outputs, and blue blocks are intermediate results.  $l$  is the minimum level that can be executed for scheme conversions.

```

1 #include "pegasus/utils.cc"
2 for(int epoch = 0; epoch < total_epochs; epoch++) {
3     // Forward Propagation
4     Dense(X, W1, b1, f1); // Layer 1
5     Relu(f1, d1);
6     Dense(d1, W2, b2, f2); // Layer 2
7     Relu(f2, d2);
8     Dense(d2, W3, b3, f3); // Layer 3
9     Softmax(f3, d3);
10    // Backward Propagation
11    Delta_Softmax(y, d3, delta3); // Layer 3
12    Drelu(f2);
13    Delta_dense(W3, delta3, f2, delta2); // Layer 2
14    Repack(delta2);
15    Drelu(f1);
16    Delta_dense(W2, delta2, f1, delta1); // Layer 1
17    Repack(delta1);
18    // Weight Updates
19    Update_Model_Dense(W1, b1, X, delta1); // Layer 1
20    Update_Model_Dense(W2, b2, d1, delta2); // Layer 2
21    Update_Model_Dense(W3, b3, d2, delta3); // Layer 3
22 }

```

Listing 1: A sample privacy-preserving training code in C++ for a multi-layer perceptron model generated by HE-SecureNet. We present only the necessary parameters of each function here.  $X$  and  $y$  are the training samples and their corresponding labels.  $W_i$  and  $b_i$  are the model weight and bias of the  $i$ -th layer.

**Network Architectures.** The models evaluated with HE-SecureNet include:

- **Logistic Regression (LR):** A linear model for binary classification.
- **Multi-layer Perceptron (MLP):** A fully connected neural network with two 128-neuron hidden layers, followed by a Softmax output layer.
- **CNN:** A convolutional neural network with one  $5 \times 5$  convolutional layer with a stride of two and two fully connected layers, where

the hidden layer between them contains 128 neurons. Each layer is followed by ReLU activation.

- **LeNet-5:** A classic CNN architecture designed for handwritten digit recognition, comprising convolutional, MaxPooling, and fully connected layers.

**Training Parameters.** The training process was conducted using stochastic gradient descent (SGD) with the following parameters:

- **Learning Rate:**  $\eta = 0.01$ .
- **Batch Size:** Varied between 128 and 1024 based on the model architecture.
- **Epochs:** We use 15 epochs for the Breast Cancer dataset and up to 25 epochs for the MNIST and CIFAR-10 datasets.

Table 5: Number of level adjustments for different models. Best values in bold.

| Model   | ConcreteML [51] | HE-SecureNet (this work) | Improvement |
|---------|-----------------|--------------------------|-------------|
| MLP     | 118,272         | <b>512</b>               | 231×        |
| CNN     | 100,205         | <b>1,080</b>             | 93×         |
| LeNet-5 | 33,986          | <b>4,836</b>             | 7×          |

Table 6: Performance Comparison for HE-based training on the Breast Cancer Dataset. Best values in bold.

| Method                   | Epoch Time (s) | Best Accuracy |
|--------------------------|----------------|---------------|
| TenSEAL [7]              | 44.9           | 62.6%         |
| ConcreteML [51]          | 232.7          | 93.9%         |
| HE-SecureNet (this work) | <b>44.6</b>    | <b>95.6%</b>  |

**Table 7: Runtime and communication for HE-based training on the MNIST Dataset. Best values in bold.**

| Models            | MLP        |                          |             | CNN        |                          |             |
|-------------------|------------|--------------------------|-------------|------------|--------------------------|-------------|
| Metrics           | Glyph [34] | HE-SecureNet (this work) | Improvement | Glyph [34] | HE-SecureNet (this work) | Improvement |
| Linear (s)        | 2,717      | <b>1,193</b>             | 2.3×        | 6,895      | <b>4,058</b>             | 1.7×        |
| Nonlinear (s)     | 2,115      | <b>1,003</b>             | 2.1×        | 3,516      | <b>1,641</b>             | 2.1×        |
| Total Latency (s) | 4,833      | <b>2,196</b>             | 2.2×        | 10,411     | <b>5,699</b>             | 1.8×        |
| Comm. (GB)        | 810        | <b>149</b>               | 5.4×        | 680        | <b>132</b>               | 5.2×        |

**Table 8: Runtime, accuracy, and communication overhead for different batch sizes. Best values in bold.**

| Dataset  | Model   | Benchmark                | Accuracy      | Batch Size=128  |              | Batch Size=256  |              | Batch Size=1024 |              |
|----------|---------|--------------------------|---------------|-----------------|--------------|-----------------|--------------|-----------------|--------------|
|          |         |                          |               | Runtime (hours) | Comm. (GB)   | Runtime (hours) | Comm. (GB)   | Runtime (hours) | Comm. (GB)   |
| MNIST    | MLP     | Keller & Sun [32]        | <b>97.8%</b>  | <b>1.25</b>     | 532.6        | <b>1.11</b>     | 713.4        | 0.98            | 991.6        |
|          |         | HE-SecureNet (this work) | <b>97.8%</b>  | 1.91            | <b>149.4</b> | 1.21            | <b>149.4</b> | <b>0.63</b>     | <b>149.4</b> |
|          | CNN     | Keller & Sun [32]        | 97.25%        | <b>1.81</b>     | 1,128.3      | <b>1.72</b>     | 1,321.4      | <b>1.62</b>     | 1,601.8      |
|          |         | HE-SecureNet (this work) | <b>98.03%</b> | 3.17            | <b>132.2</b> | 2.22            | <b>132.2</b> | 1.77            | <b>132.2</b> |
|          | LeNet-5 | Keller & Sun [32]        | 98.1%         | <b>5.17</b>     | 3,976.6      | <b>3.49</b>     | 4,100.5      | <b>3.11</b>     | 4,634.8      |
|          |         | HE-SecureNet (this work) | <b>98.5%</b>  | 11.34           | <b>70.6</b>  | 10.71           | <b>70.6</b>  | 10.23           | <b>70.6</b>  |
| CIFAR-10 | LeNet-5 | Keller & Sun [32]        | 67.64%        | <b>8.31</b>     | 8,092.6      | <b>8.28</b>     | 8,167.1      | <b>8.19</b>     | 8,256.3      |
|          |         | HE-SecureNet (this work) | <b>68.22%</b> | 15.95           | <b>190.5</b> | 15.1            | <b>190.5</b> | 14.4            | <b>190.5</b> |

## 5.2 Ciphertext Level Management

Table 5 shows that HE-SecureNet’s level management strategy significantly reduces the number of level adjustments required for various models. Specifically, up to 230× of level adjustments are eliminated among three models. These results underline the effectiveness of level management in optimizing model performance while minimizing computational complexity.

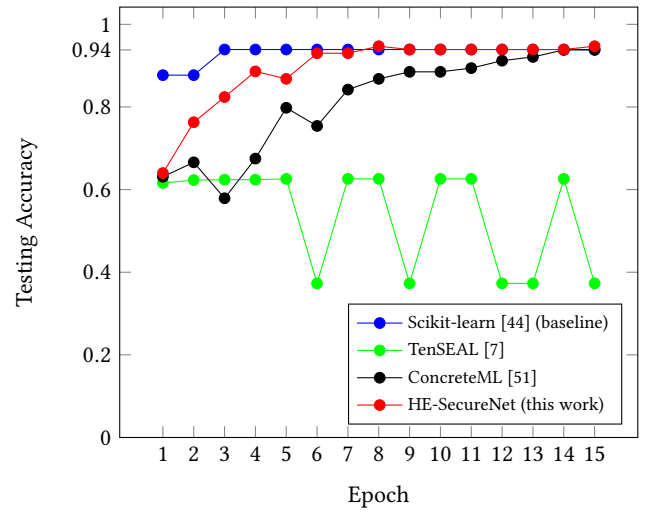
## 5.3 Comparisons with HE-based Approaches

**5.3.1 Logistic Regression on Breast Cancer Dataset.** We compare HE-SecureNet against TenSEAL [7] and ConcreteML [51].

**Performance.** As shown in Table 6, HE-SecureNet achieves a 5.2× faster training time compared to ConcreteML, while improving model accuracy by 33% over TenSEAL with similar training time. The speedup is attributed to HE-SecureNet’s CKKS batching mechanism, which simultaneously processes multiple samples in linear functions. The improved accuracy is due to ConcreteML’s reliance on quantization and TenSEAL’s polynomial approximations of non-linear functions.

**Accuracy.** Figure 3 illustrates HE-SecureNet’s faster convergence compared to TenSEAL and ConcreteML. Moreover, HE-SecureNet’s accuracy closely aligns with the plaintext results obtained from Scikit-learn [44].

**5.3.2 CNN Training on MNIST Dataset.** We compare HE-SecureNet with Glyph [34], which combines BGV [11] for linear computations and TFHE [18] for non-linear evaluations in Table 7. HE-SecureNet, using the CKKS [17]-FHEW [20] hybrid scheme from

**Figure 3: Accuracy curves of logistic regression models on the Breast Cancer Dataset. Higher is better.**

PEGASUS [35], achieves a 2× speedup in training time across all architectures. The performance improvement is attributed to HE-SecureNet’s optimized scheme-switching and LUT evaluations. Regarding communication costs, HE-SecureNet achieves 5× lower communication overhead than Glyph [34].

HE-SecureNet’s hybrid scheme effectively reduces computation latency for both linear and non-linear operations, as well as communication overhead, making it significantly faster than Glyph. Both HE-SecureNet and Glyph achieve the same accuracy of 98.7% on MNIST; however, HE-SecureNet converges more quickly during training.

## 5.4 Comparison with MPC-based Approaches

Table 8 compares HE-SecureNet’s runtime and communication overhead with the MPC-based framework of Keller & Sun [32] on the MNIST and CIFAR-10 datasets. Their work performs floating-point training in the two-party setting. Unlike MPC, HE-SecureNet maintains a constant communication overhead across all batch sizes, achieving up to 65.7× lower communication costs. Furthermore, HE-SecureNet consistently outperforms [32] in terms of model accuracy. While increasing the batch size accelerates HE-SecureNet’s training process and narrows the performance gap between MPC- and HE-based methods, HE-SecureNet remains up to 3.3× slower than [32] due to the computational overhead of HE. However, recent advancements in hardware technologies, such as the collaboration between Duality Technologies and Intel<sup>1</sup> to optimize HE on Intel’s 3rd Gen Xeon processors, have high potential to substantially accelerate HE computations. With predictions of up to 1000× acceleration in HE over the next five years, HE-SecureNet will become an increasingly practical solution for private ML training. By leveraging these hardware-driven optimizations, HE-SecureNet allows ML developers to build and train private models without requiring in-depth knowledge of cryptography.

## 6 Conclusion and Future Work

In this paper, we presented HE-SecureNet, a Homomorphic Encryption (HE)-based framework for privacy-preserving training of ML models. Leveraging the Pegasus library [35], HE-SecureNet efficiently integrates CKKS for linear computations and FHEW for non-linear evaluations, achieving secure, non-interactive model training with a single server while maintaining strong data privacy. By automatically converting ONNX models into privacy-preserving C++ training code, HE-SecureNet allows developers without cryptographic expertise to use secure machine learning techniques.

Although we address data derived from a specific client, HE-SecureNet can be extended to scenarios where the training data is sourced from multiple parties through multi-key HE [14, 15]. In the multi-client framework, each data holder encrypts their data with their respective public keys before uploading the training data to the cloud, where the server can still perform HE computations on the encrypted data for model training. Subsequently, the resultant model will be stored in the cloud, transitioning directly to a PPML inference service.

## Acknowledgment

This project received funding from the European Research Council (ERC) under the European Union’s research and innovation programs Horizon 2020 (PSOTI/850990) and Horizon Europe (PRIV-TOOLS/101124778). It was co-funded by the Deutsche Forschungsgemeinschaft (DFG) within SFB 1119 CROSSING/236615297 and by

the National Science and Technology Council in Taiwan (112-2917-I-002-010).

## References

- [1] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/> Software available from tensorflow.org.
- [2] Joseph Abrera. 2024. Data Privacy and Security in Cloud Computing: A Comprehensive Review. *Journal of Computer Science and Information Technology* (2024).
- [3] Nitin Agrawal, Ali Shahin Shamsabadi, Matt J Kusner, and Adrià Gascón. 2019. QUOTIENT: Two-Party Secure Neural Network Training and Prediction. In *CCS*.
- [4] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. 2025. General functional bootstrapping using CKKS. In *ICC*.
- [5] Sebastian Becker, Christoph Bösch, Benjamin Hettwer, Thomas Hoeren, Merlin Rombach, Sven Trieflinger, and Hossein Yalame. 2025. Multi-Party Computation in Corporate Data Processing: Legal and Technical Insights. *Cryptology ePrint Archive* 2025/463 (2025).
- [6] Yaniv Ben-Itzhak, Helen Möllering, Benny Pinkas, Thomas Schneider, Ajith Suresh, Oleksandr Tkachenko, Shay Vargaftik, Christian Weinert, Hossein Yalame, and Avishay Yanai. 2024. ScionFL: Efficient and Robust Secure Quantized Aggregation. In *IEEE SaTML*.
- [7] Ayoub Benaissa, Bilal Retiat, Bogdan Cebere, and Alaa Eddine Belfedhal. 2021. TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption. In *ICLR Workshop on Distributed and Private Machine Learning*.
- [8] Song Bian, Zhou Zhang, Haowen Pan, Ran Mao, Zian Zhao, Yier Jin, and Zhenyu Guan. 2023. HE3DB: An Efficient and Elastic Encrypted Database via Arithmetic-And-Logic Fully Homomorphic Encryption. In *CCS*.
- [9] Fabian Boemer, Rosario Cammarota, Daniel Demmler, Thomas Schneider, and Hossein Yalame. 2020. MP2ML: A Mixed-Protocol Machine Learning Framework for Private Inference. In *ARES*.
- [10] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. 2020. CHIMERA: Combining Ring-LWE-based Fully Homomorphic Encryption Schemes. *Journal of Mathematical Cryptology* (2020).
- [11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Transactions on Computation Theory* (2014).
- [12] Andreas Brügemann, Robin Hundt, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2023. FLUTE: Fast and Secure Lookup Table Evaluations. In *IEEE S&P*.
- [13] Andreas Brügemann, Oliver Schick, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2024. Don’t Eject The Impostor: Fast Three-Party Computation With A Known Cheater. In *IEEE S&P*.
- [14] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Multi-Key Homomorphic Encryption from TFHE. In *ASIACRYPT*.
- [15] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. 2019. Efficient Multi-Key Homomorphic Encryption with Packed Ciphertexts with Application to Oblivious Neural Network Inference. In *CCS*.
- [16] Jung Hee Cheon, Kyoohyun Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *EUROCRYPT*. Springer.
- [17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *ASIACRYPT*.
- [18] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption over the Torus. *Journal of Cryptology* (2020).
- [19] Yann Disser, Daniel Günther, Thomas Schneider, Maximilian Stillger, Arthur Wigand, and Hossein Yalame. 2023. Breaking the Size Barrier: Universal Circuits Meet Lookup Tables. In *ASIACRYPT*.
- [20] Léo Ducas and Daniele Micciancio. 2015. FHEW: Bootstrapping Homomorphic Encryption in Less Than A Second. In *EUROCRYPT*.
- [21] Vasisht Duddu, Anudeep Das, Nora Khayata, Hossein Yalame, Thomas Schneider, and N Asokan. 2024. Attesting Distributional Properties of Training Data for Machine Learning. In *ESORICS*.
- [22] Farshad Firouzi, Shiyi Jiang, Krishnendu Chakraborty, Bahar Farahani, Mahmoud Daneshmand, Jaeseung Song, and Kunal Mankodiya. 2022. Fusion of IoT, AI, Edge-Fog-Cloud, and Blockchain: Challenges, Solutions, and a Case Study in Healthcare and Medicine. *IEEE Internet of Things Journal* (2022).

<sup>1</sup><https://dualitytech.com/partners/intel/>

- [23] Till Gehlhar, Felix Marx, Thomas Schneider, Ajith Suresh, Tobias Wehrle, and Hossein Yalame. 2023. SafeFL: MPC-Friendly Framework for Private and Robust Federated Learning. In *DLS@IEEE S&P*.
- [24] Daniel Günther, Joachim Schmidt, Thomas Schneider, and Hossein Yalame. 2024. FLUENT: A Tool For Efficient Mixed-Protocol Semi-Private Function Evaluation. In *ACSAC*.
- [25] Bulbul Gupta, Pooja Mittal, and Tabish Mufti. 2021. A Review on Amazon Web Service (AWS), Microsoft Azure & Google Cloud Platform (GCP) Services. In *Proceedings of the 2nd International Conference on ICT for Digital, Smart, and Sustainable Development, ICIDSSD 2020, 27-28 February 2020, Jamia Hamdard, New Delhi, India*. 9.
- [26] Christopher Harth-Kitzrow, Ajith Suresh, Yongqin Wang, Hossein Yalame, Georg Carle, and Murali Annaram. 2025. High-Throughput Secure Multi-party Computation with an Honest Majority in Various Network Settings. *PETS* (2025).
- [27] Aditya Hegde, Helen Möllering, Thomas Schneider, and Hossein Yalame. 2021. SoK: Efficient Privacy-Preserving Clustering. *PETS* (2021).
- [28] Junyuan Hong, Lingjuan Lyu, Jiayu Zhou, and Michael Spranger. 2022. Outsourcing Training without Uploading Data via Efficient Collaborative Open-Source Sampling. *NeurIPS* (2022).
- [29] Somayeh Iranpak, Asadollah Shahbahrani, and Hassan Shakeri. 2021. Remote Patient Monitoring and Classifying using the Internet of Things Platform Combined with Cloud Computing. *Journal of Big Data* (2021).
- [30] Rafia Islam, Vardhan Patamsetti, Aparna Gadhi, Ragha Madhavi Gondu, Chinna Manikanta Bandaru, Sai Chaitanya Kesani, and Olatunde Abiona. 2023. The Future of Cloud Computing: Benefits and Challenges. *International Journal of Communications, Network and System Sciences* (2023).
- [31] Hannah Keller, Helen Möllering, Thomas Schneider, and Hossein Yalame. 2021. Balancing Quality and Efficiency in Private Clustering with Affinity Propagation. *SECRYPT* (2021).
- [32] Marcel Keller and Ke Sun. 2022. Secure Quantized Training for Deep Learning. In *ICML*.
- [33] Tzu-Li Liu, Yu-Te Ku, Ming-Chien Ho, Feng-Hao Liu, Ming-Ching Chang, Chih-Fan Hsu, Wei-Chao Chen, and Shih-Hao Hung. 2023. An Efficient CKKS-FHEW/TFHE Hybrid Encrypted Inference Framework. In *ESORICS*.
- [34] Qian Lou, Bo Feng, Geoffrey Charles Fox, and Lei Jiang. 2020. Glyph: Fast and Accurately Training Deep Neural Networks on Encrypted Data. In *NeurIPS*.
- [35] Wen Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. 2021. PEGASUS: Bridging Polynomial and Non-Polynomial Evaluations in Homomorphic Encryption. In *IEEE S&P*.
- [36] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *EUROCRYPT*.
- [37] Felix Marx, Thomas Schneider, Ajith Suresh, Tobias Wehrle, Christian Weinert, and Hossein Yalame. 2023. WW-FL: Secure and Private Large-Scale Federated Learning. *arXiv preprint arXiv:2302.09904* (2023).
- [38] Microsoft. 2025. ONNX Runtime. <https://www.onnxruntime.ai>.
- [39] Payman Mohassel and Yupeng Zhang. 2017. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In *IEEE S&P*.
- [40] Jean-Pierre Münch, Thomas Schneider, and Hossein Yalame. 2021. VASA: Vector AES Instructions for Security Applications. In *ACSAC*.
- [41] Karthik Nandakumar, Nalini Ratha, Sharath Pankanti, and Shai Halevi. 2019. Towards Deep Neural Network Training on Encrypted Data. In *CVPR*.
- [42] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *arXiv preprint arXiv:1912.01703* (2019).
- [43] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2021. ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation. In *USENIX Security*.
- [44] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research* (2011).
- [45] Oded Regev. 2009. On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *J. ACM* (2009).
- [46] Thomas Schneider, Huan-Chih Wang, and Hossein Yalame. 2025. HE-SecureNet: An Efficient and Usable Framework for Model Training via Homomorphic Encryption. In *WPES*.
- [47] Thomas Schneider, Hossein Yalame, and Michael Yonli. 2023. Griffin: Towards Mixed Multi-Key Homomorphic Encryption. *SECRYPT* (2023).
- [48] Shailja Tripathi. 2023. Digital Transformation and Cloud Computing: System Dynamics Modeling Approach. *International Journal For Multidisciplinary Research* (2023).
- [49] Steven M Williamson and Victor Prybutok. 2024. Balancing Privacy and Progress: a Review of Privacy Challenges, Systemic Oversight, and Patient Perceptions in AI-Driven Healthcare. *Applied Sciences* (2024).
- [50] Jiaqi Xue, Yancheng Zhang, Yanshan Wang, Xueqiang Wang, Hao Zheng, and Qian Lou. 2024. CryptoTrain: Fast Secure Training on Encrypted Dataset. In *CCS-LAMPS*.
- [51] Zama. 2022. Concrete ML: A Privacy-Preserving Machine Learning Library using Fully Homomorphic Encryption for Data Scientists. <https://github.com/zama-ai/concrete-ml>.
- [52] Yancheng Zhang, Mengxin Zheng, Yuzhang Shang, Xun Chen, and Qian Lou. 2024. HEPRUNE: Fast Private Training of Deep Neural Networks with Encrypted Data Pruning. *NeurIPS* (2024).
- [53] Zhou Zhang, Song Bian, Zian Zhao, Ran Mao, Haoyi Zhou, Jiafeng Hua, Yier Jin, and Zhenyu Guan. 2024. ArcEDB: An Arbitrary-Precision Encrypted Database via (Amortized) Modular Homomorphic Encryption. In *CCS*.